

Operating System:

Abraham Silberschatz, Peter Baer Galvin, Greg Gagne Operating System Principles, 8th edition Wiley- India,2011.

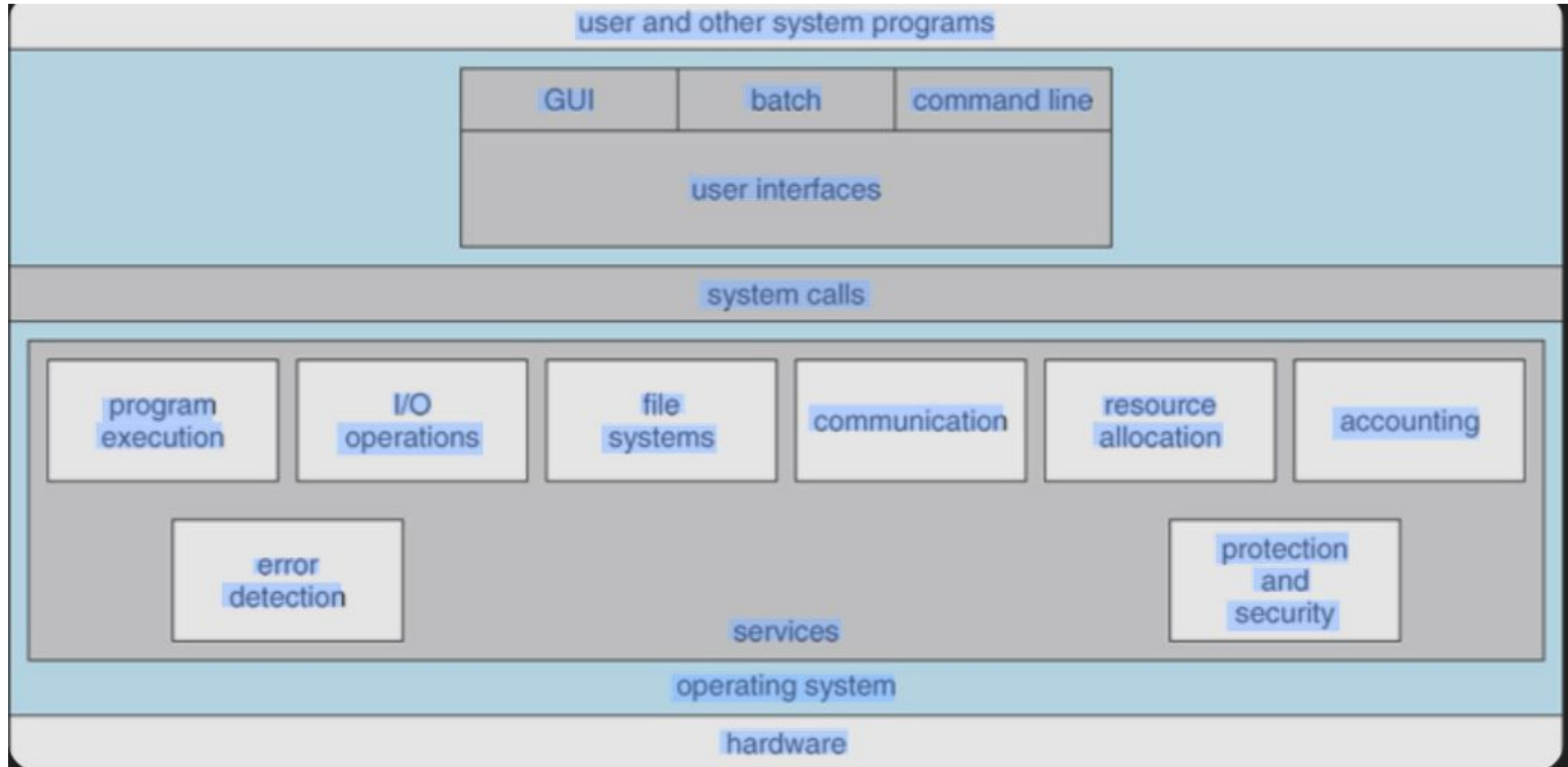
**Shubha Malige
Asst.Professor
CSE(Cyber Security)**

Operating System Services

Operating System Services

- ✓ Provides an environment for the execution of programs.
- ✓ Provides certain services to:
 - i. programs
 - ii. Users of those programs
- ✓ Basically, two types of services:
 - ** Set of operating-system services provides functions that are helpful to the user.
 - **Set of operating-system functions for ensuring the efficient operation of the system itself via resource sharing

Operating System Services



Operating System Services

- ✓ An operating system provides an environment for the execution of programs.
- ✓ It provides certain services to programs and to the users of those programs.
- ✓ The specific services provided, of course, differ from one operating system to another, but we can identify common classes.
- ✓ These operating-system services are provided for the convenience of the programmer, to make the programming task easier.
- ✓ The above fig shows various operating system services and how they interrelate.

Operating System Services

OS Services Helpful to the user

User Interface—Almost all operating systems have a user Interface(UI). Several forms:

Command-Line(CLI)—uses text commands and a method for entering them(say, a keyboard for typing in commands in a specific format with specific options)

Graphics User Interface(GUI)—The interface is a window system with a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text..

Batch Interface—Commands and directives to control those commands are entered into files, and those files are executed.

Some systems provide two or all 3 variations.

Operating System Services

OS Services helpful to the user:

Program Execution-The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally(indicating error).

I/O operations—A running program may require I/O, which may involve a file or an I/O device.

File-System Manipulation-Programs need to read and write files and directories, create and delete them, search them, list file information, permission management

Operating System Services

OS Services Helpful to the user(Contd.)

- ✓ Communications—Processes may exchange information, on the same computer or between computers over a network.
 - **Communications may be via shared memory or through message passing(packets moved by the OS)
- ✓ Error Detection—OS needs to be constantly aware of possible errors.
 - ** May occur in the CPU and memory hardware, in I/O devices, in user program
 - ** For each type of error, OS should take the appropriate action to ensure correct and consistent computing.
 - ** Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Operating System Services

OS Services Ensuring Efficient Operation

- ✓ Resource allocation—When multiple users or multiple jobs are running concurrently, resources must be allocated to each of them.
 - **Many types of resources— CPU cycles, main memory, file storage, I/O devices
- ✓ Accounting—To keep track of which users use how much and what kinds of computer resources.
- ✓ Protection and Security—The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other.
 - **Protection involves ensuring that all access to system resources is controlled.
 - ** Security of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

User Operating System Interfaces

One approach for os interface is command-line interface, or Command interpreter, that allows users to directly enter commands to be performed by the operating system.

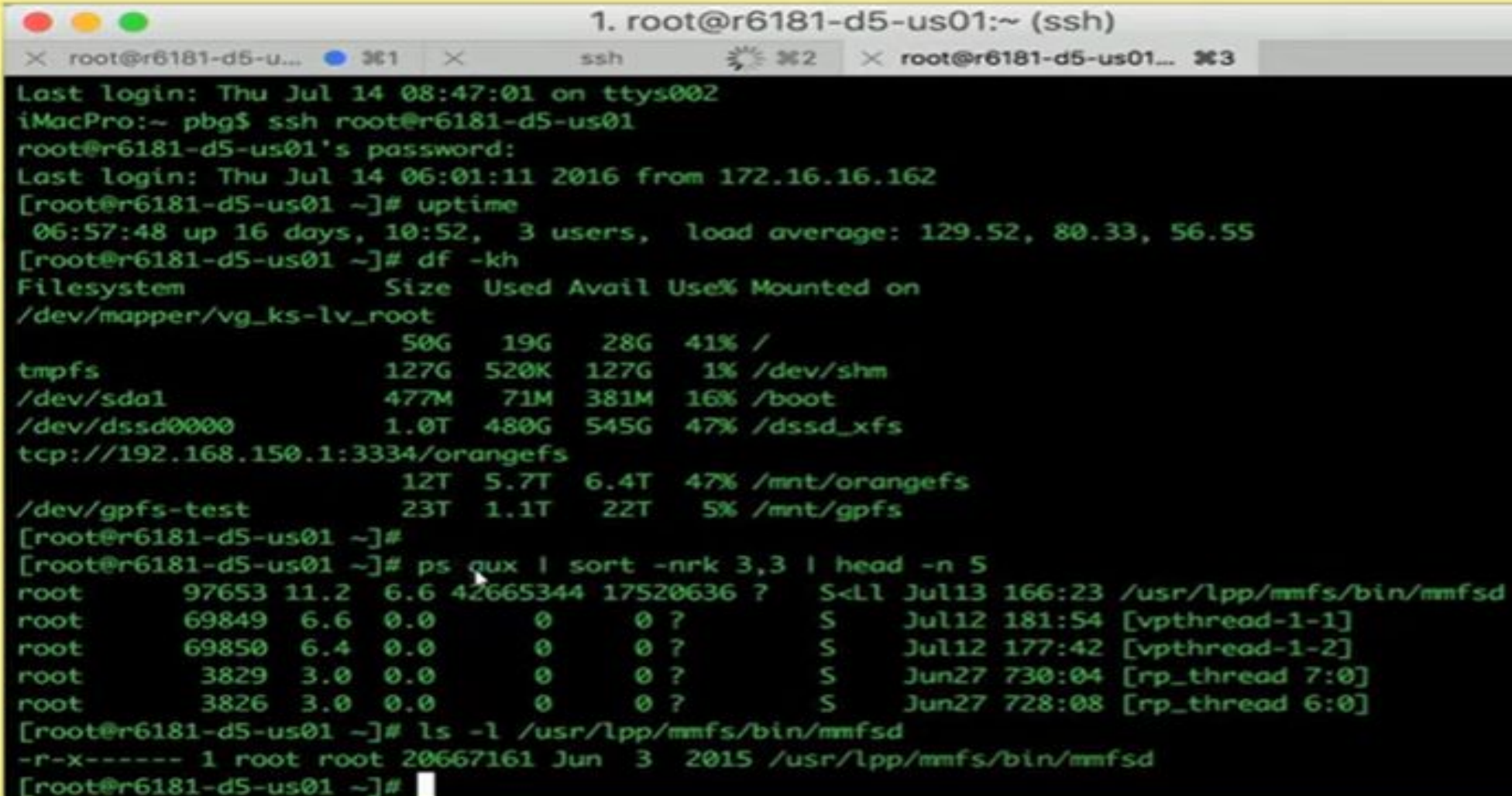
The other allows users to interface with the operating system via a graphical user interface, or GUI.

Command Interpreters: (CLI)

- ✓ CLI allows users to directly enter commands to be performed by the operating system.
- ✓ Some operating systems include the command interpreter in the kernel.
- ✓ Some operating systems, such as Windows and Unix, treat the command interpreter as a special program that is running when a job is initiated or when a user first logs on.
- ✓ On systems with multiple command interpreters to choose from the interpreters are known as Shells.
- ✓ The main function of the command interpreter is to get and execute the next user specified command.
- ✓ Sometimes command built-in, sometimes just names of programs
 - ** If the latter, adding new features doesn't require shell modification.

User Operating System Interfaces

The Bourne shell command interpreter in Solaris

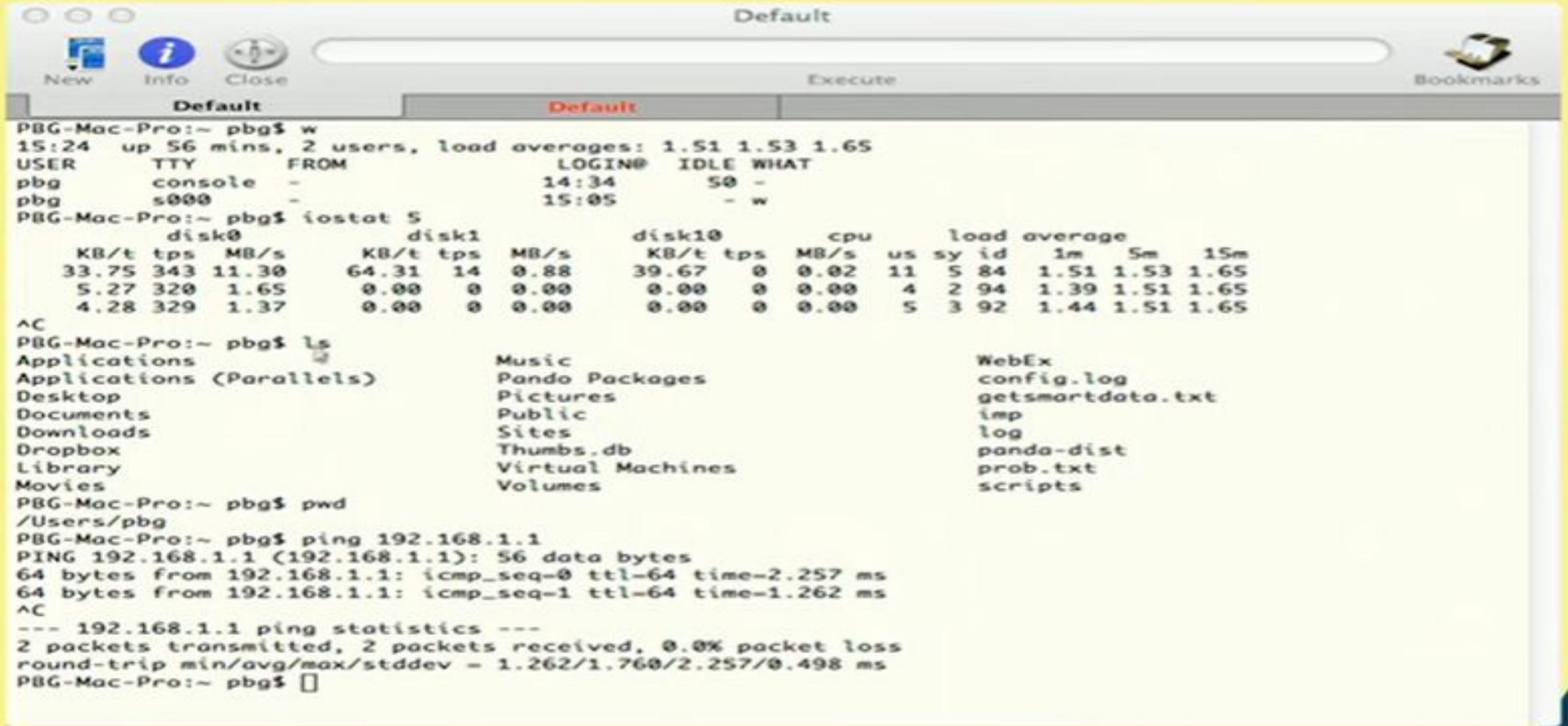


The screenshot shows a terminal window titled "1. root@r6181-d5-us01:~ (ssh)". The window contains the following text:

```
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root    50G       19G   28G   41% /
tmpfs                      127G       520K  127G    1% /dev/shm
/dev/sda1                   477M       71M   381M   16% /boot
/dev/dssd0000               1.0T      480G   545G   47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs 12T       5.7T   6.4T   47% /mnt/orangefs
/dev/gpfs-test              23T       1.1T   22T    5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root      97653 11.2  6.6 42665344 17520636 ?    S<Ll  Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root      69849  6.6  0.0      0      0 ?    S     Jul12 181:54 [vpthread-1-1]
root      69850  6.4  0.0      0      0 ?    S     Jul12 177:42 [vpthread-1-2]
root      3829   3.0  0.0      0      0 ?    S     Jun27 730:04 [rp_thread 7:0]
root      3826   3.0  0.0      0      0 ?    S     Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x----- 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#
```

User Operating System Interfaces

Bourne Shell Command Interpreter



```
PBG-Mac-Pro:~ pbg$ w
15:24 up 56 mins, 2 users, load averages: 1.51 1.53 1.65
USER      TTY      FROM          LOGIN@   IDLE   WHAT
pbg       console  -             14:34    50    -
pbg       s000    -             15:05    -    w

PBG-Mac-Pro:~ pbg$ iostat 5

            disk0            disk1            disk10            cpu            load average
  KB/t tps  MB/s    KB/t tps  MB/s    KB/t tps  MB/s  us sy id  1m   5m   15m
   33.75 343 11.30    64.31 14   0.88    39.67  0   0.02  11  5 84   1.51 1.53 1.65
    5.27 320  1.65     0.00  0   0.00     0.00  0   0.00   4  2 94   1.39 1.51 1.65
    4.28 329  1.37     0.00  0   0.00     0.00  0   0.00   5  3 92   1.44 1.51 1.65

^C
PBG-Mac-Pro:~ pbg$ ls
Applications          Music                  WebEx
Applications (Parallels) Pando Packages        config.log
Desktop               Pictures               getsmartdata.txt
Documents             Public                 imp
Downloads             Sites                  log
Dropbox               Thumbs.db              panda-dist
Library               Virtual Machines       prob.txt
Movies                Volumes                scripts

PBG-Mac-Pro:~ pbg$ pwd
/Users/pbg

PBG-Mac-Pro:~ pbg$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: icmp_seq=0 ttl=64 time=2.257 ms
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.262 ms
^C
--- 192.168.1.1 ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 1.262/1.760/2.257/0.498 ms
PBG-Mac-Pro:~ pbg$
```

User Operating System Interfaces

Graphical User Interfaces(GUI)

- ✓ User-friendly desktop metaphor interface

 - && Usually mouse, keyboard and monitor

 - && Icons represent files, programs, actions etc

 - && Various mouse buttons over objects in the interface cause various actions(Provide information, options, execute function, open directory(known as a folder)

 - && Invented at Xerox PARC

- ✓ Many systems now include both CLI and GUI interfaces

 - && Microsoft Window is GUI with CLI “command” shell

 - && Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available.

 - && Unix and Linux have CLI with optional GUI interfaces(CDE, KDE, GNOME)

User Operating System Interfaces

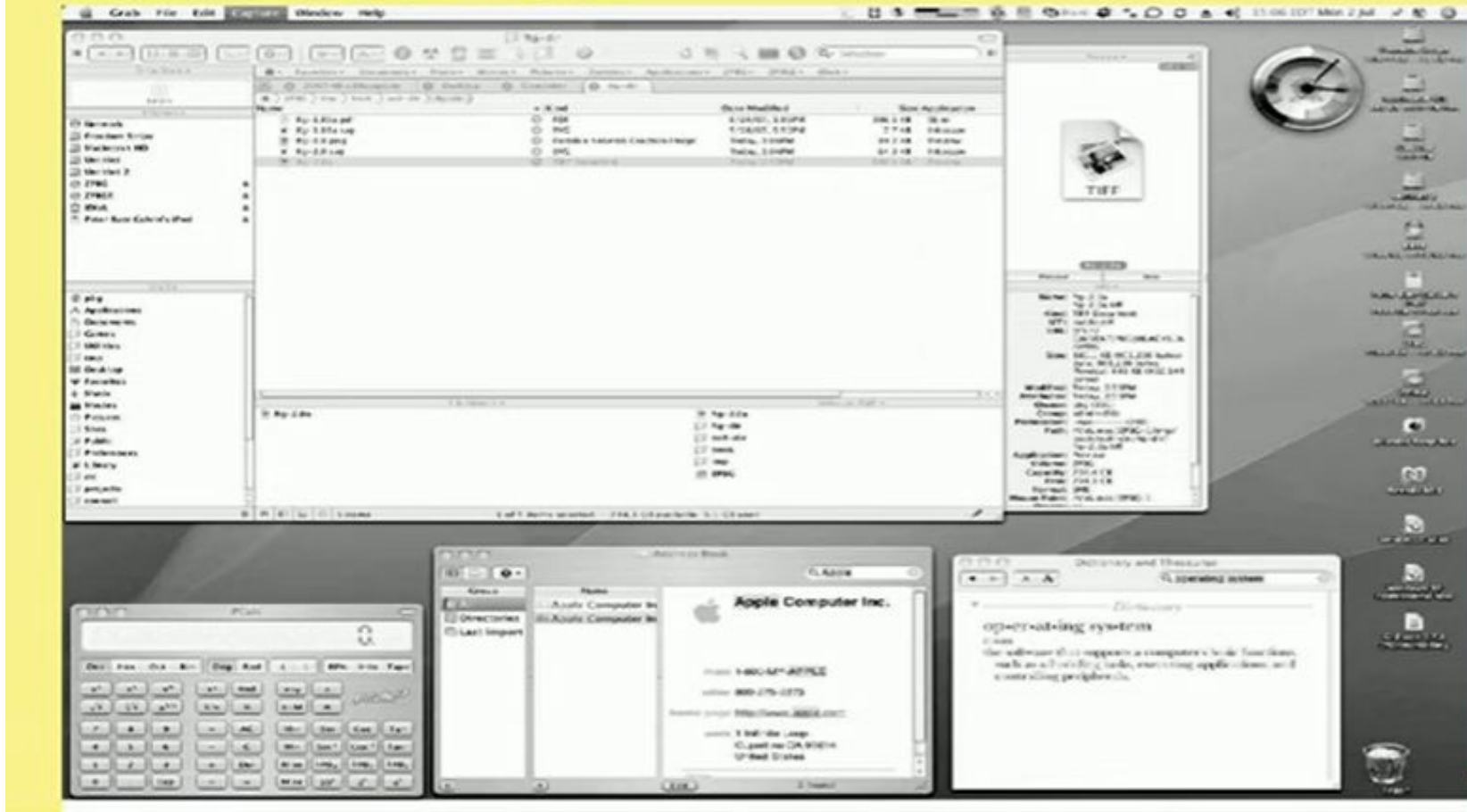
Touchscreen Interfaces

- ? Touchscreen devices require new interfaces
 - ? Mouse not possible or not desired
 - ? Actions and selection based on gestures
 - ? Virtual keyboard for text entry
- ? Voice commands.



User Operating System Interfaces

The Mac OS X GUI



System Calls

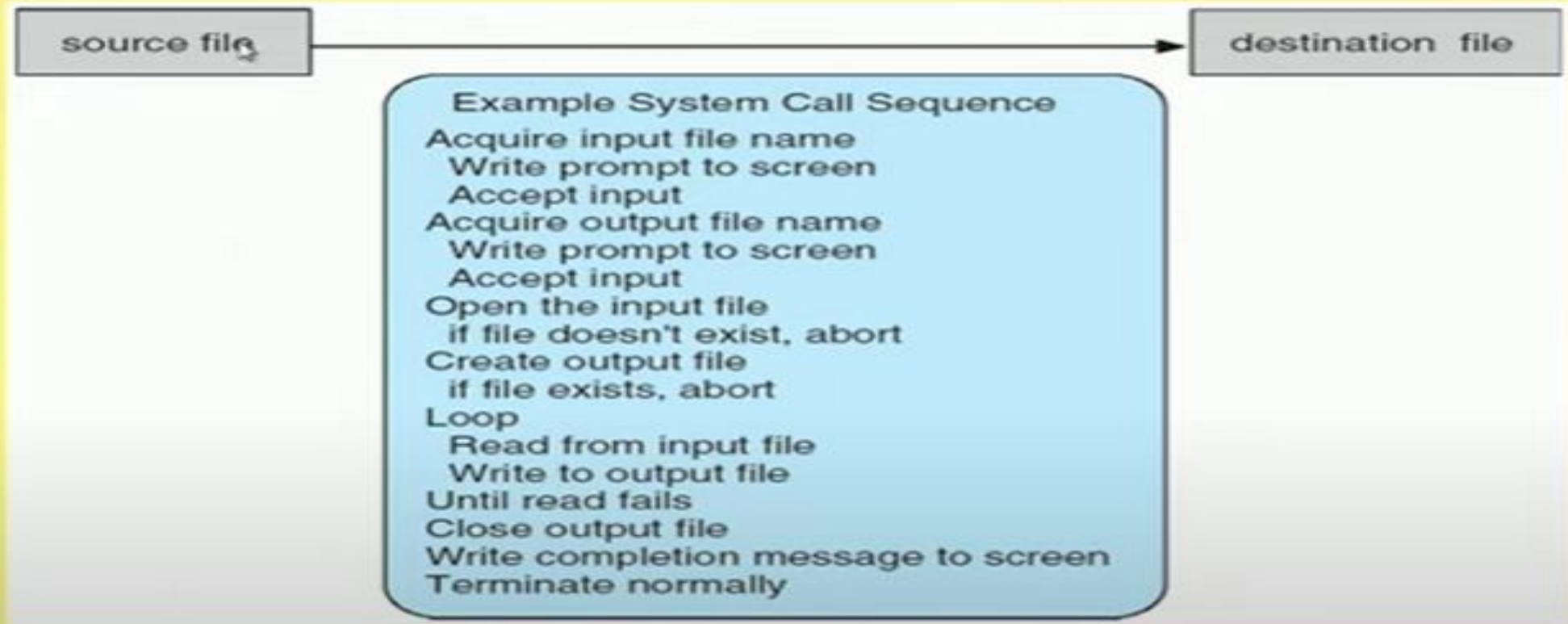
System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call
- Three most common APIs are:
 - Win32 API for Windows,
 - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X),
 - Java API for the Java virtual machine (JVM)

System Calls

Example of System Calls

- System call sequence to copy the contents of one file



System Calls

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the man page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count)
```

return value function name parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

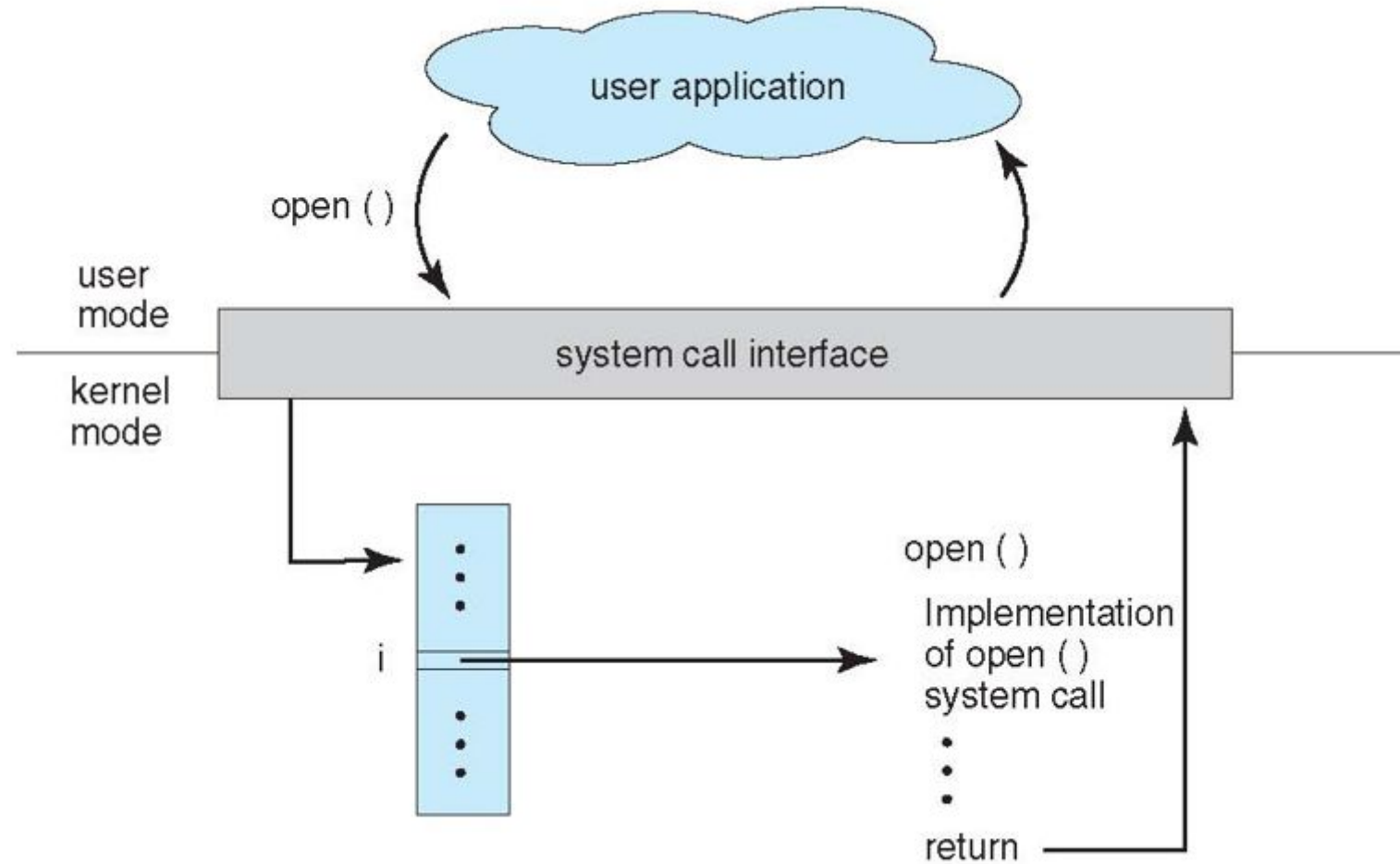
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

System Call Implementation

- Typically, a number associated with each system call
 - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call
 - Most details of OS interface hidden from programmer by API
 - ▶ Managed by run-time support library (set of functions built into libraries included with compiler)

API-System Call OS Relationship



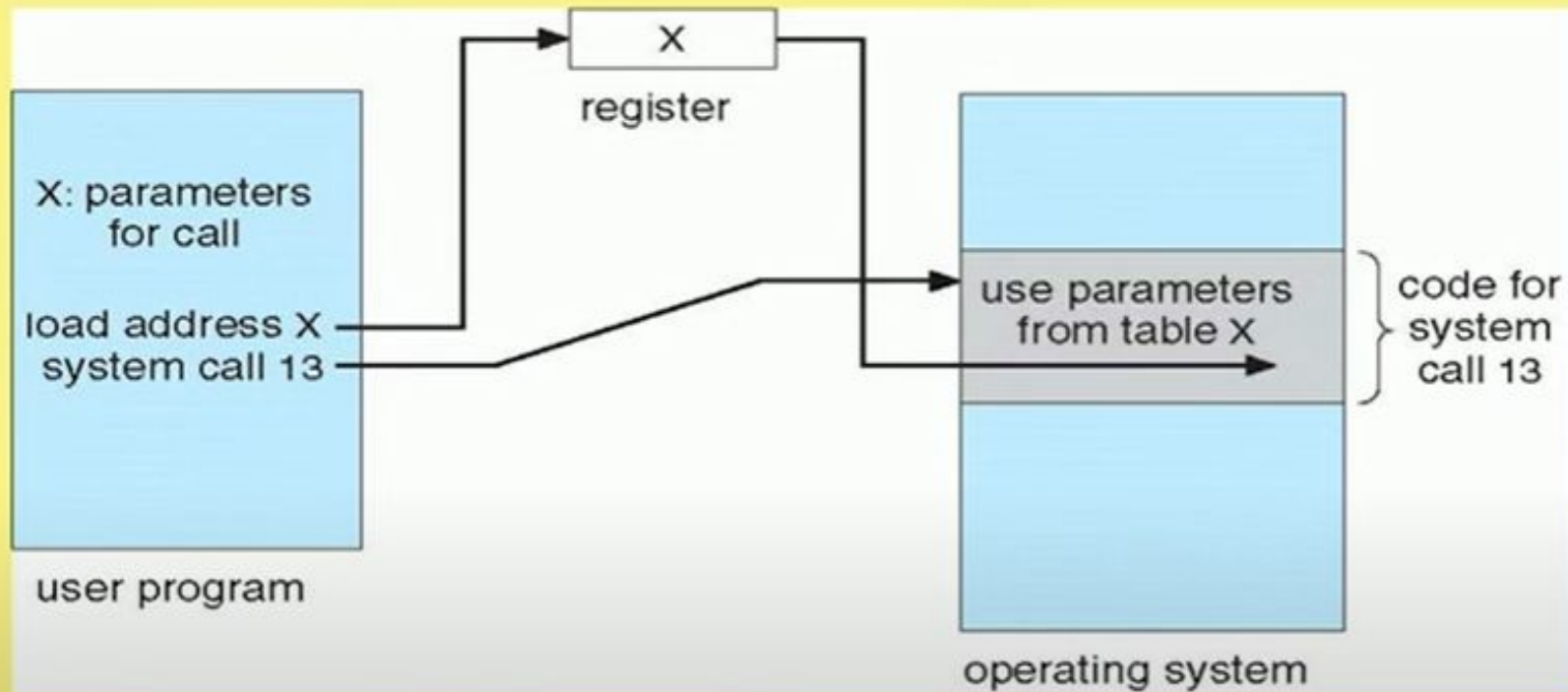
System Call parameter Passing

- ❑ Often, more information is required than simply identity of desired system call
 - ❑ Exact type and amount of information vary according to OS and call
- ❑ Three general methods used to pass parameters to the OS
 - ❑ Simplest: pass the parameters in registers
 - ▶ In some cases, may be more parameters than registers
 - ❑ Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
 - ▶ This approach taken by Linux and Solaris
 - ❑ Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
 - ❑ Block and stack methods do not limit the number or length of parameters being passed

System Calls

Parameter Passing via Table

x points to a block of parameters. x is loaded into a register



Types of system calls

Types of System Calls

- System calls can be grouped roughly into six major categories
 - Process control,
 - File manipulation,
 - Device manipulation,
 - Information maintenance,
 - Communications,
 - Protection.

Types of system calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Types of system calls

System Calls – Process Control

- create process, terminate process
- end, abort
- load, execute
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory
- Dump memory if error
- **Debugger** for determining **bugs, single step** execution
- **Locks** for managing access to shared data between processes

Types of system calls

System Calls – File Management

- Create file
- Delete file
- Open and Close file
- Read, Write, Reposition
- Get and Set file attributes

Types of system calls

System Calls – Device Management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

Types of system calls

System Calls -- Information Maintenance

- get time or date,
- set time or date
- get system data,
- set system data
- get and set process, file, or device attributes

Types of system calls

System Calls – Communications

- create, delete communication connection
- if **message passing model**:
 - send, receive message
 - To **host name** or **process name**
 - From **client** to **server**
- If **shared-memory model**:
 - create and gain access to memory regions
- transfer status information
- attach and detach remote devices

Types of system calls

System Calls -- Protection

- Control access to resources
- Get and set permissions
- Allow and deny user access

Types of system calls

EXAMPLE OF STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call(s) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program. This is shown in Figure 2.9.

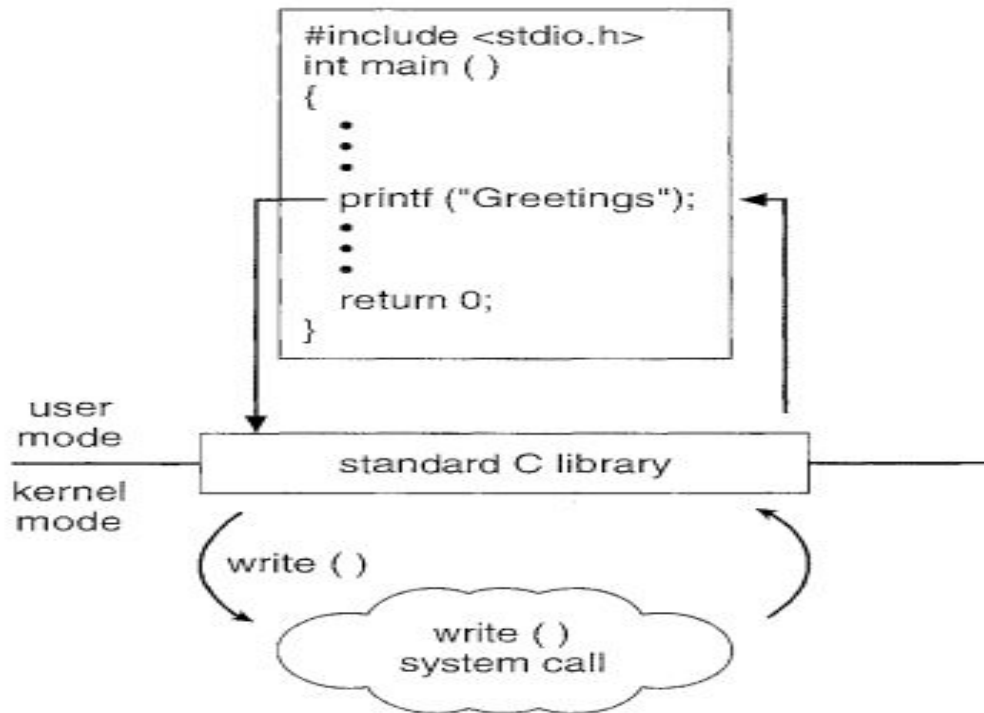


Figure 2.9 Standard C library handling of `write()`.

System Boot

✓ When power initialized on system, execution starts at a fixed memory location

** Firmware ROM used to hold initial boot code

✓ Operating system must be made available to hardware so hardware can start it

** Small piece of code-**bootstrap loader**, stored in ROM or EEPROM locates the kernel, loads it into memory, and starts it.

** Sometimes two step process where boot block at fixed location loaded by ROM code, which loads bootstrap loader from disk.

✓ Common bootstrap loader, GRUB, allows selection of kernel from multiple disks, versions, kernel options

✓ Kernel loads and system is then running.

Operating System Structures

A modern operating system is divided into small, well-defined modules instead of being built as one large monolithic system. Each module has clearly defined inputs, outputs, and specific functions, making the system easier to design, manage, and modify.

Integration into the Kernel – These individual modules are interconnected and combined to form the operating system kernel. Proper organization and interaction among components ensure that the entire system functions correctly and efficiently.

Operating System Structures

Simple Structure:

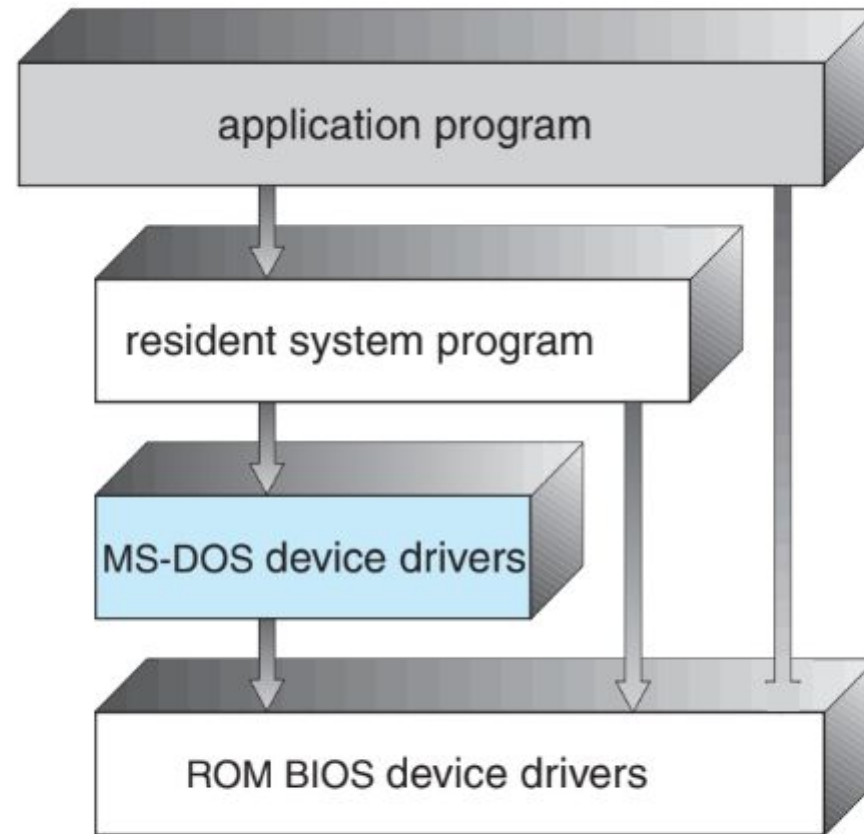


Figure 2.11 MS-DOS layer structure.

Operating System Structures

Simple Structure:

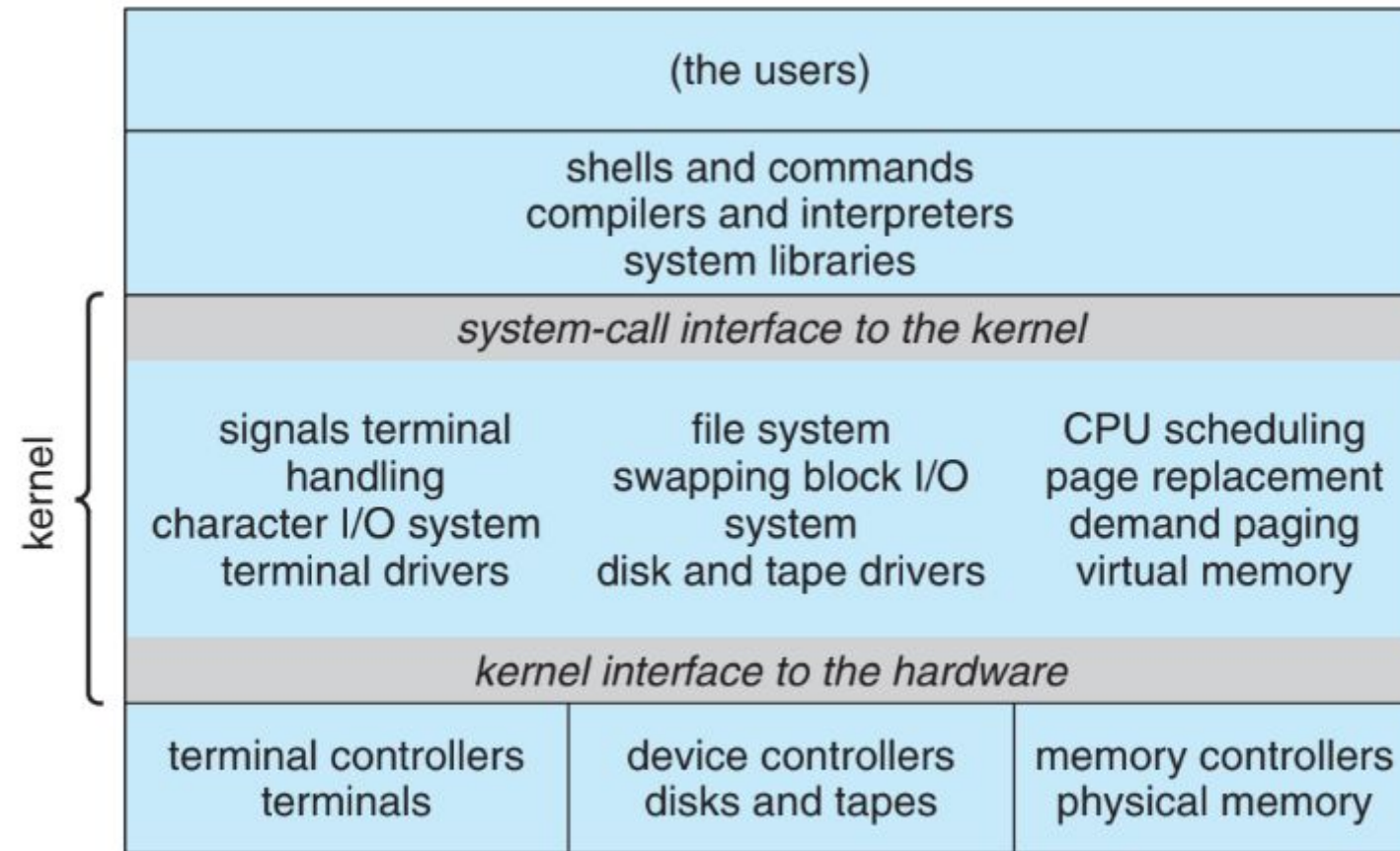


Figure 2.12 Traditional UNIX system structure.

Operating System Structures

Layered Approach:

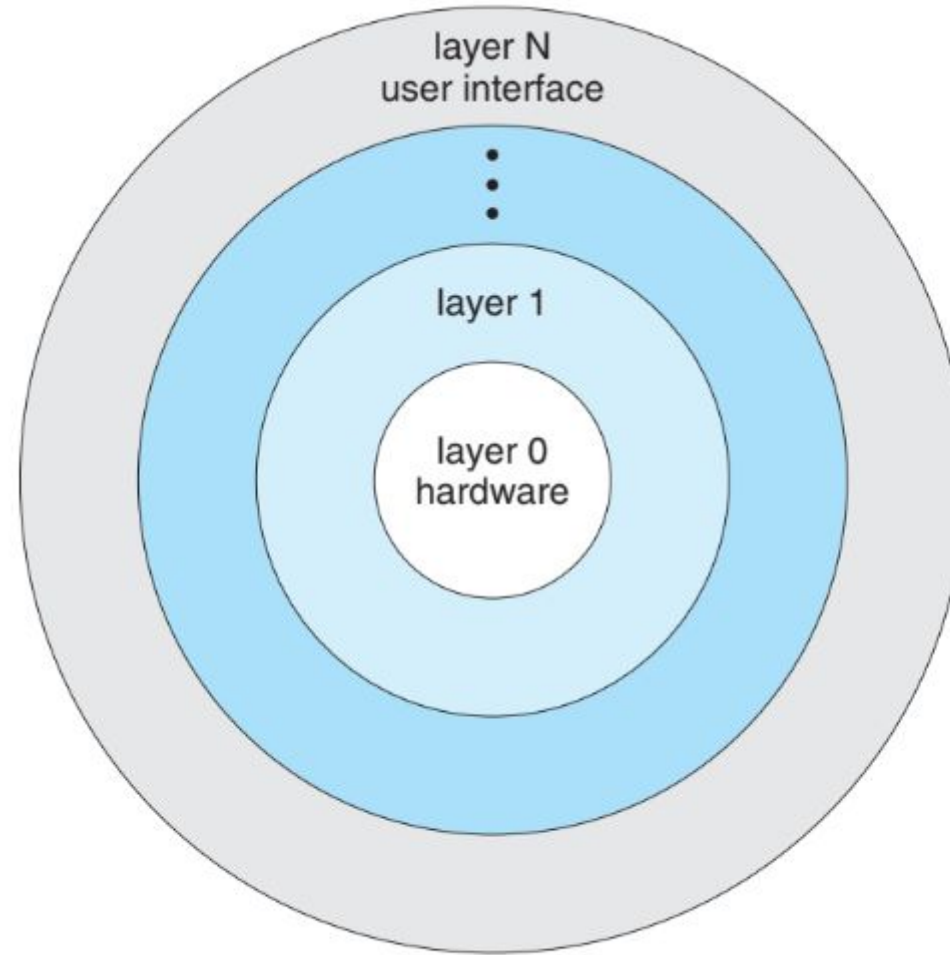


Figure 2.13 A layered operating system.

Operating System Structures

Microkernel Approach:

The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space. Communication is provided through **message passing**, which was described in Section 2.4.5. For example, if the client program wishes to access a file, it

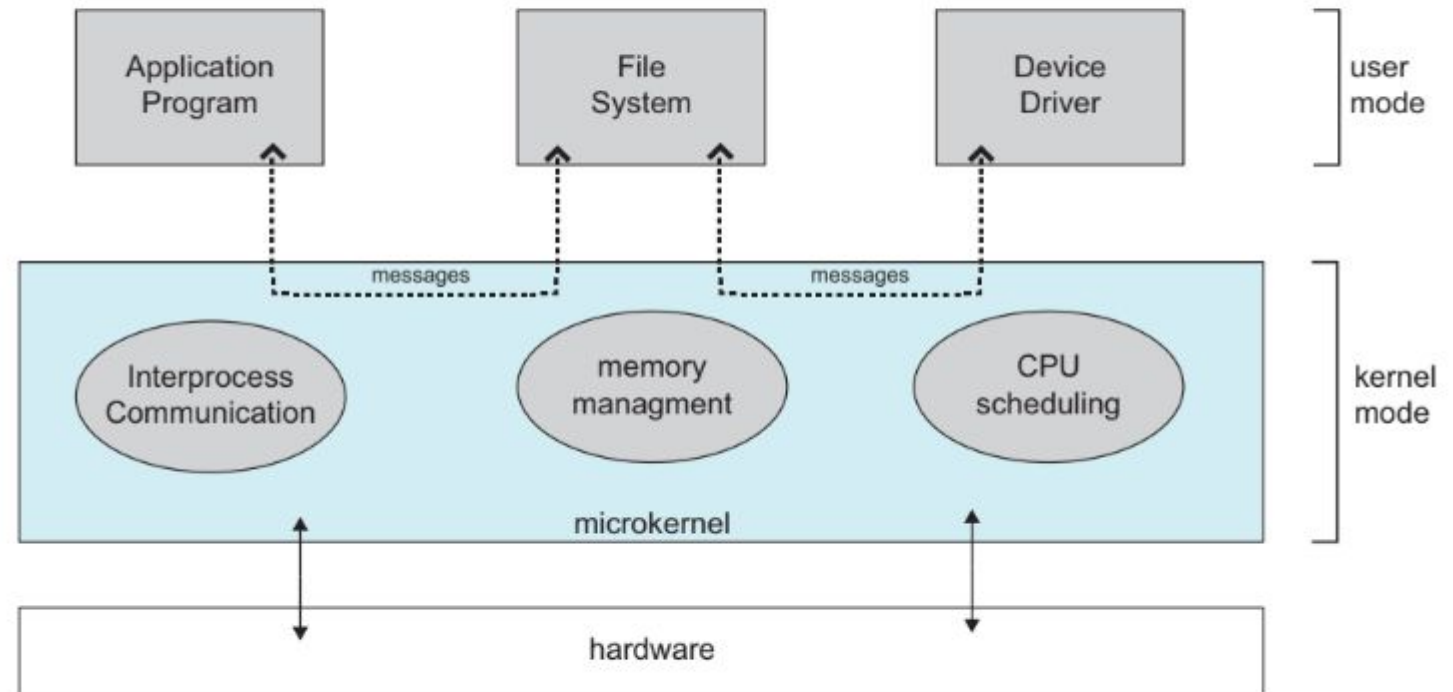


Figure 2.14 Architecture of a typical microkernel.

Operating System Structures

Modules:

2.7 Operating-System Structure

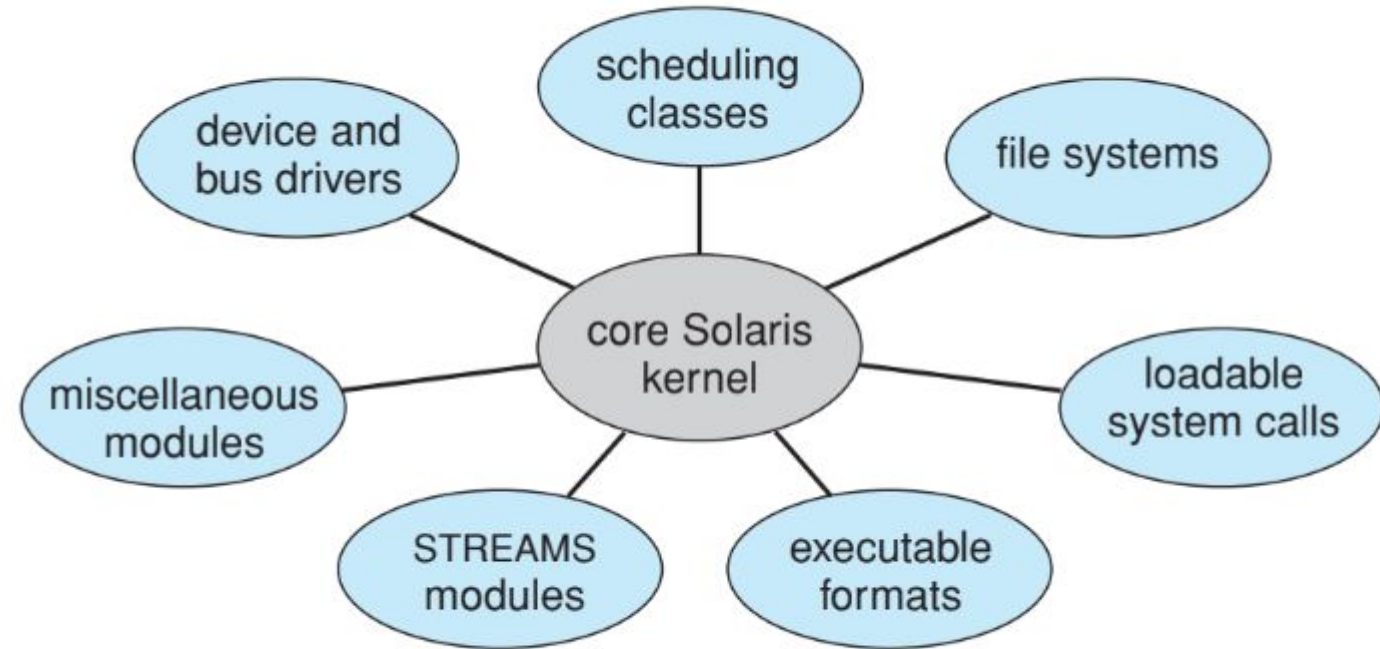


Figure 2.15 Solaris loadable modules.

Operating System Structures

Modules:

The Solaris operating system structure, shown in Figure 2.15, is organized around a core kernel with seven types of loadable kernel modules:

1. Scheduling classes
2. File systems
3. Loadable system calls
4. Executable formats
5. STREAMS modules
6. Miscellaneous
7. Device and bus drivers

Operating System Structures

Hybrid Systems:

Mac OSx

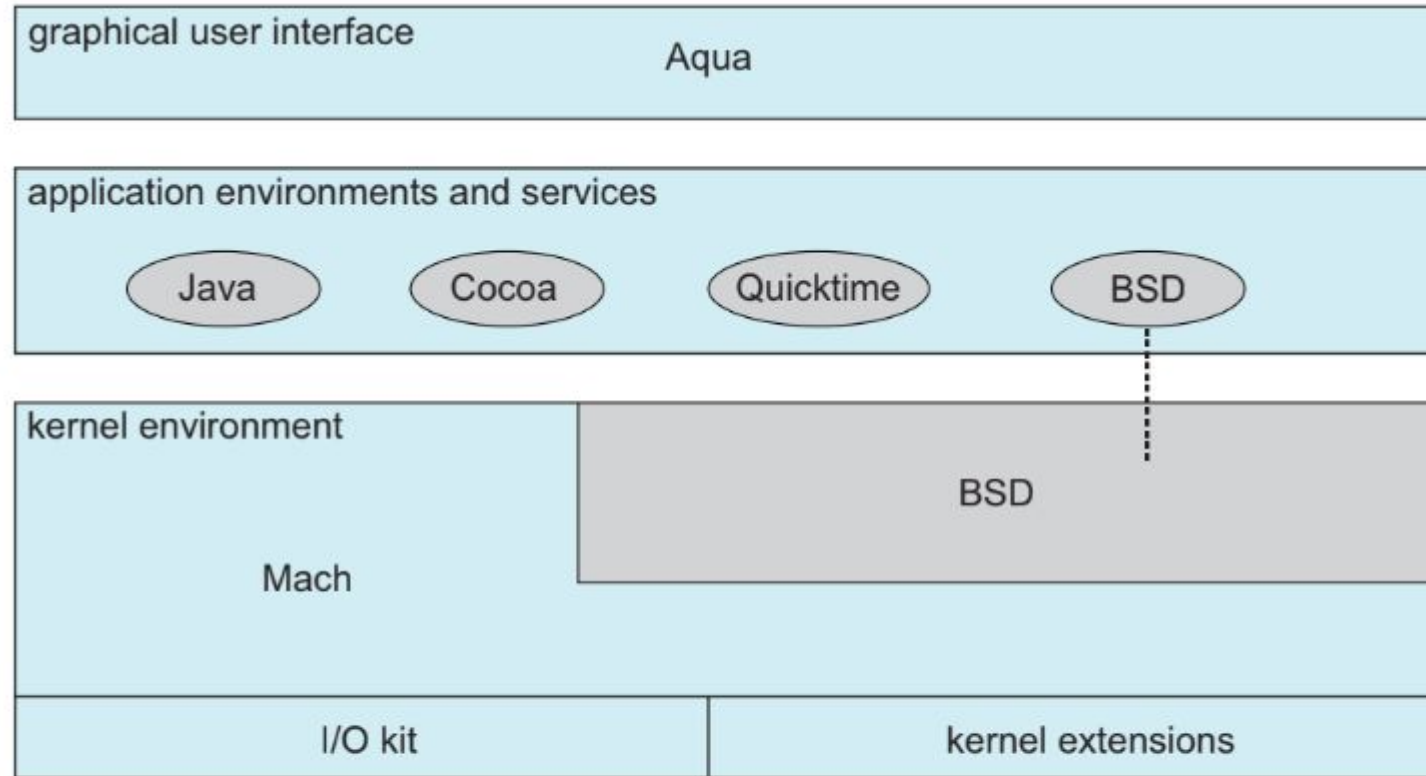


Figure 2.16 The Mac OS X structure.

Operating System Structures

Hybrid Systems:

iOS



Figure 2.17 Architecture of Apple's iOS.

Operating System Structures

Hybrid Systems:

Google's Android

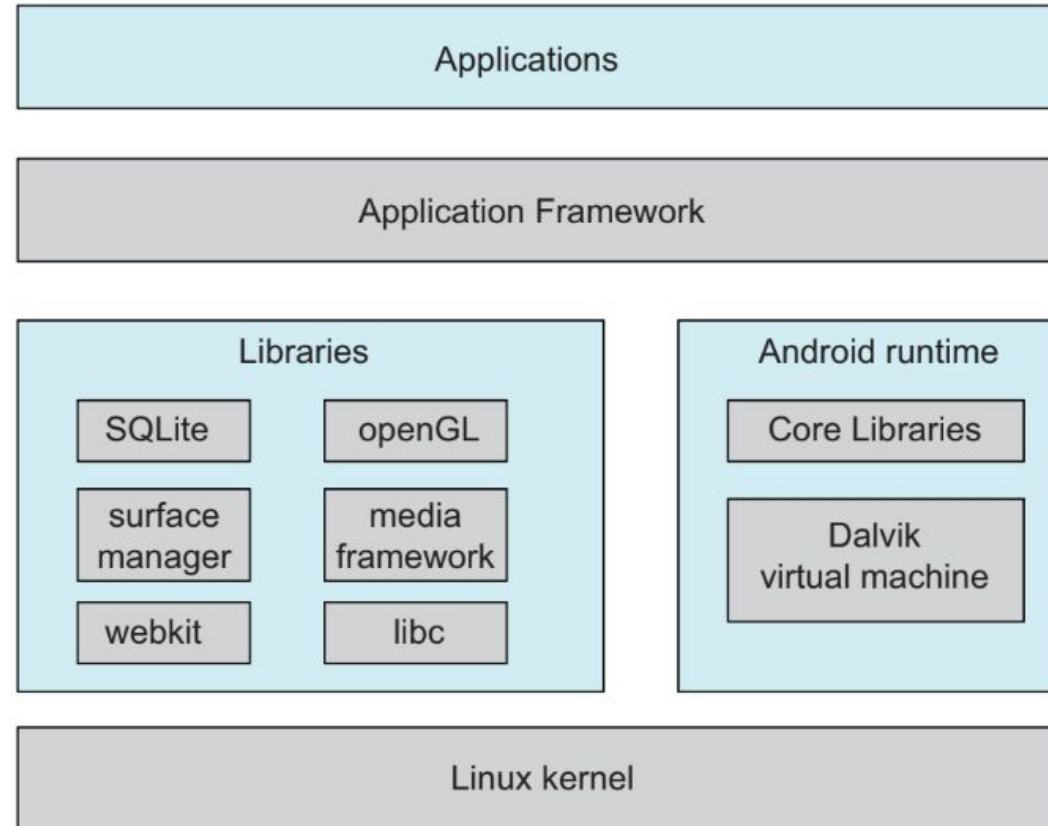


Figure 2.18 Architecture of Google's Android.

Thank You